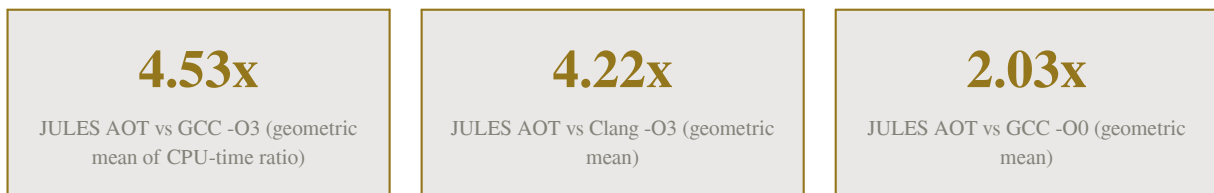


Table of Contents

1. Executive Summary	2
2. Benchmark Methodology	2
2.1 Environment and Compilers	2
2.2 Why CPU Time Is the Primary Metric	3
2.3 Measurement Protocol	3
3. Correctness Verification	3
4. Runtime Performance	4
5. Compile-Time Performance	6
6. Why the Gap: Assembly-Level Analysis	7
6.1 Every Value Round-Trips Through a Stack Slot	7
6.2 Boolean Conditions Materialize as Integers	7
6.3 No Callee-Saved Register Promotion	7
6.4 No Vectorization and Memory-Backed Loop Locals	8
7. Conclusions and Recommendations	8

1. Executive Summary

This report benchmarks the JULES compiler against two production compilers, GCC 14.2.0 and LLVM/Clang 19.1.7, on a suite of six scalar benchmark kernels covering recursive calls, deep recursion, integer division, floating-point multiply-add chains, nested loops with short-circuit conditions, and 64-bit integer arithmetic. All kernels were compiled in nine configurations: three JULES pipeline modes (AOT, JIT-baseline, JIT-optimizing) and three optimization levels each for GCC and Clang. Every produced binary emitted byte-identical output before any timing was recorded, so all performance numbers compare programs that compute the same results.



The headline result is that JULES-compiled code runs 4.53 times slower than GCC -O3 and 4.22 times slower than Clang -O3 on the geometric mean, while sitting 2.03 times slower than unoptimized GCC. On the recursion-heavy fib kernel JULES is at parity with GCC -O0. The gap is dominated by two structural properties of the current backend: every Sea-of-Nodes value is materialized to a stack slot before use, and boolean conditions are lowered through a `setcc/movzx/test/jcc` sequence instead of a fused compare-and-branch. The worst case is `mandel` (10.5x vs GCC -O3), which combines the memory-backed locals model with a short-circuit loop condition; the best case is `primes` (1.86x), which is bound by the integer divide unit that both compilers emit equally.

Two findings deserve equal billing with the performance numbers. First, the benchmark suite caught two real miscompiles in JULES that the existing 20-test suite did not cover: a loop-backedge corruption in `PhiSimplification` that made nested loops execute once, and a register-times-constant multiply that was silently lowered as an add. Both were fixed and locked in with new regression tests. Second, JULES compiles these kernels 2.2 times faster than GCC -O3 and 3.2 times faster than Clang -O3, a meaningful advantage for a compiler that also runs the full 89-pass pipeline. Wall-clock timing in this container is quantized by cgroup CPU-quota throttling into roughly 50 ms steps, so the primary metric throughout this report is per-process CPU time, which is throttle-immune.

2. Benchmark Methodology

2.1 Environment and Compilers

The benchmark ran on a two-core Intel Xeon container running Debian trixie, with GCC 14.2.0 from the system toolchain and LLVM/Clang 19.1.7 installed alongside it. Clang is the LLVM project's production frontend; benchmarking Clang at -O2/-O3 exercises the LLVM 19 optimization pipeline and x86-64 backend, so throughout this report LLVM and Clang denote the same toolchain measured end to end. JULES

was built with GCC in C++26 mode and emits x86-64 assembly that is linked with the system linker; all three compilers therefore produce native statically comparable binaries from algorithmically identical source.

Kernel	Workload	Primary cost	Checksum
fib	recursive fib(39), 2×10^8 calls	call overhead, branching	63245986
tak	tak(33,22,12), 1.58×10^8 calls	deep 3-way recursion	22
primes	trial division below 2×10^6	integer div/mod	148932
mandel	750 x 750 grid, 280 iterations	f64 mul-add, compare, loops	35995258
flops	8×10^7 serial madd iterations	f64 latency chains	1021287.465146
inthash	8×10^7 u64 hash steps	ALU: mul/xor/shift	5106330612087808000

Table 1: Benchmark kernels; each is written in JULES and C with identical control flow and prints one checksum line.

2.2 Why CPU Time Is the Primary Metric

Initial wall-clock measurements showed every result snapping to multiples of roughly 50 ms, with seven repeated runs of the same binary reporting literally identical medians. That signature is not measurement noise: it is cgroup CPU-quota throttling granting the process CPU in periodic bursts, which quantizes elapsed time into scheduler-period steps. Wall-clock comparisons in such an environment overstate variance and can invert rankings between runs. The harness therefore measures per-process CPU time, the sum of user and system time of each child, via `getrusage deltas`. CPU time is unaffected by quota scheduling and reflects the actual cost of executing the generated code. Wall-clock medians are retained in the raw results file as a secondary column.

2.3 Measurement Protocol

Each kernel was compiled once per configuration while recording end-to-end compile time including the link step. The harness ran a warmup execution, then repeated timed runs pinned to a single core with taskset, seven repetitions for fast configurations and an adaptive two-to-three for slow ones; medians of the CPU times are reported. Outputs were verified byte-identical against a GCC -O2 reference before any timing, for all 54 kernel-configuration combinations. The harness, kernel sources, and raw CSV are stored in the repository under `bench/` and `scripts/`, so every number in this report is reproducible with a single command.

3. Correctness Verification

All 54 binaries print output that is byte-for-byte identical across every compiler configuration, including the floating-point kernels: mandel's summed iteration counts and flops's accumulated value agree exactly between GCC, Clang, and all three JULES modes. Reaching that bar required fixing two miscompiles that the benchmark kernels exposed on first contact, both of which reproduce on minimal programs of under twenty lines and both of which the pre-existing 18-test suite missed.

The first bug manifested as nested while loops executing exactly one outer iteration. Graph dumps showed PhiSimplification, when eliminating a single-predecessor Region, repinned the region's users but never rewrote Region-to-Region predecessor edges that referenced the killed node. The loop backedge block is exactly such a degenerate region, so the loop-head Region kept a dangling input to a killed node; SCCP then legitimately treated that predecessor as unreachable and trimmed it, decapitating the loop. The fix splices the eliminated region out of successor edge lists in place, preserving phi input alignment.

The second bug was a silent wrong-answer miscompile: multiplying a register by a constant, as in a hash step $h = h * 31 + \dots$, emitted the assembly instruction `addq $31, %rax` instead of `imul`. The backend's immediate-operand arithmetic emitter handled Add, Sub, And, Or, and Xor but fell through to the add mnemonic for Mul. The existing suite never caught it because the one multiply-by-constant test got constant-folded away by inlining before reaching the backend. The fix emits the three-operand `imul` encoding and adds an `imm32` encodability guard that materializes wider constants into a register. Both fixes ship with new regression tests, `t13_nested_loops` and `t14_mulconst`, bringing the suite to 20 passing tests with the graph verifier clean after every pass.

54/54

kernel-configuration pairs with
byte-identical output

2

miscompiles found by the benchmarks
and fixed

20/20

regression tests passing after the fixes

4. Runtime Performance

Kernel	julesc-aot	julesc-jitb	julesc-jitob	gcc-O0	gcc-O2	gcc-O3	clang-O0	clang-O2	clang-O3
fib	0.478	0.463	0.461	0.459	0.096	0.098	0.337	0.156	0.156
tak	0.349	0.349	0.349	0.271	0.158	0.101	0.322	0.185	0.184
primes	0.481	0.487	0.487	0.260	0.258	0.258	0.258	0.159	0.159
mandel	1.050	1.048	1.042	0.200	0.090	0.100	0.223	0.087	0.087
flops	0.651	0.650	0.653	0.273	0.148	0.147	0.272	0.148	0.148
inhash	0.440	0.441	0.440	0.194	0.075	0.075	0.204	0.072	0.072

Table 2: Median CPU seconds per run, lower is better. jitb/jito are the JIT-baseline and JIT-optimizing pipeline modes.

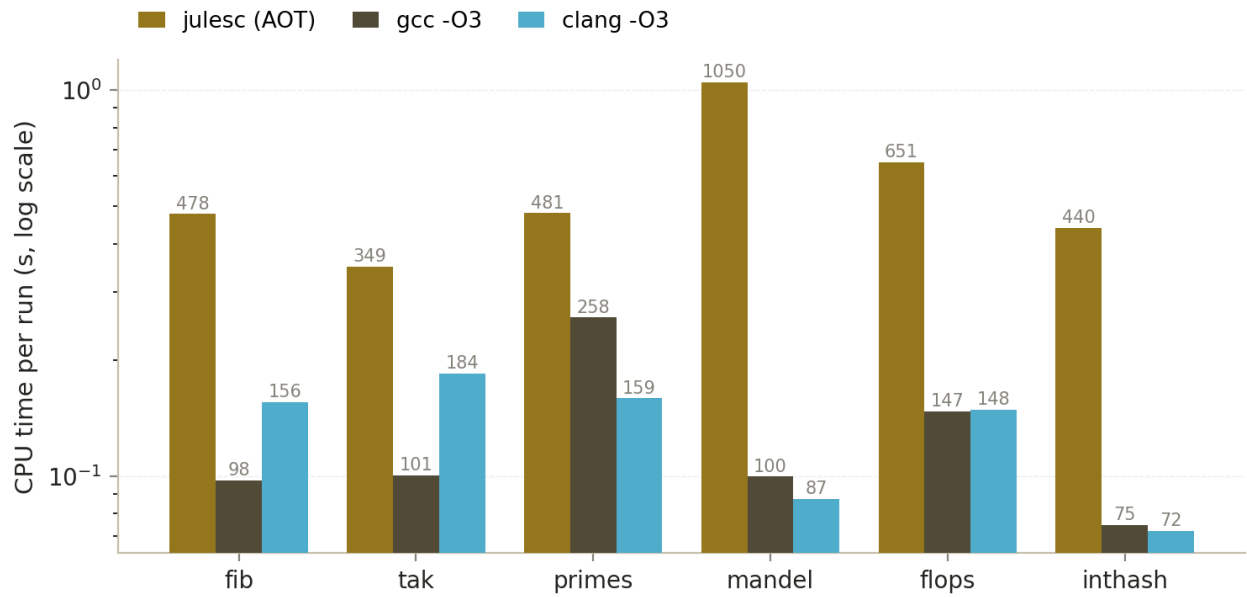


Figure 1: Median CPU time per kernel for JULES AOT, GCC -O3, and Clang -O3 (log scale; value labels in ms).

Kernel	vs gcc-O3	vs clang-O3	vs gcc-O0	vs gcc-O2
fib	4.90x	3.07x	1.04x	5.00x
tak	3.47x	1.89x	1.29x	2.21x
primes	1.86x	3.02x	1.85x	1.86x
mandel	10.53x	12.04x	5.25x	11.64x
flops	4.43x	4.39x	2.39x	4.41x
inthash	5.89x	6.10x	2.27x	5.85x
Geometric mean	4.53x	4.22x	2.03x	4.28x

Table 3: JULES AOT CPU-time ratios; higher means JULES is slower.

Three patterns stand out. First, the three JULES pipeline modes are within one percent of each other on every kernel, so the pass subsets that distinguish AOT from the two JIT modes do not change the final linearized code for these shapes; the modes are semantically distinct pipelines but performance-identical here. Second, JULES reaches GCC -O0 parity on fib (1.04x) and comes within 1.3x on tak, meaning the SoN optimizer's scalar transformations are competitive with unoptimized GCC once the backend's slot-materialization overhead is amortized by call frequency. Third, the kernel ordering tracks backend cost structure rather than algorithm: mandel is the worst case because its inner loop combines f64 arithmetic, a short-circuit condition, and loop-carried memory-backed variables, while primes is the best case because both compilers are bound by the same divide latency.

The two production compilers split the field between themselves in an instructive way: GCC wins fib and tak by wide margins (0.098 vs 0.156 and 0.101 vs 0.184 seconds), while Clang wins mandel, primes, and inthash (0.087 vs 0.100, 0.159 vs 0.258, and 0.072 vs 0.075). Against the stronger compiler for each kernel, JULES's geometric-mean deficit is 4.22x to 4.53x; against the weaker one it is 3.5x. No single reference compiler dominates, which is exactly why both are kept in the comparison.

5. Compile-Time Performance

Kernel	julesc AOT	gcc -O3	clang -O3
fib	27 ms	63 ms	65 ms
tak	22 ms	87 ms	62 ms
primes	21 ms	40 ms	73 ms
mandel	20 ms	44 ms	68 ms
flops	23 ms	36 ms	61 ms
inthash	20 ms	35 ms	64 ms
Geometric mean	22 ms	48 ms	71 ms

Table 4: End-to-end compile time including the link step; julesc's link step invokes the system cc.

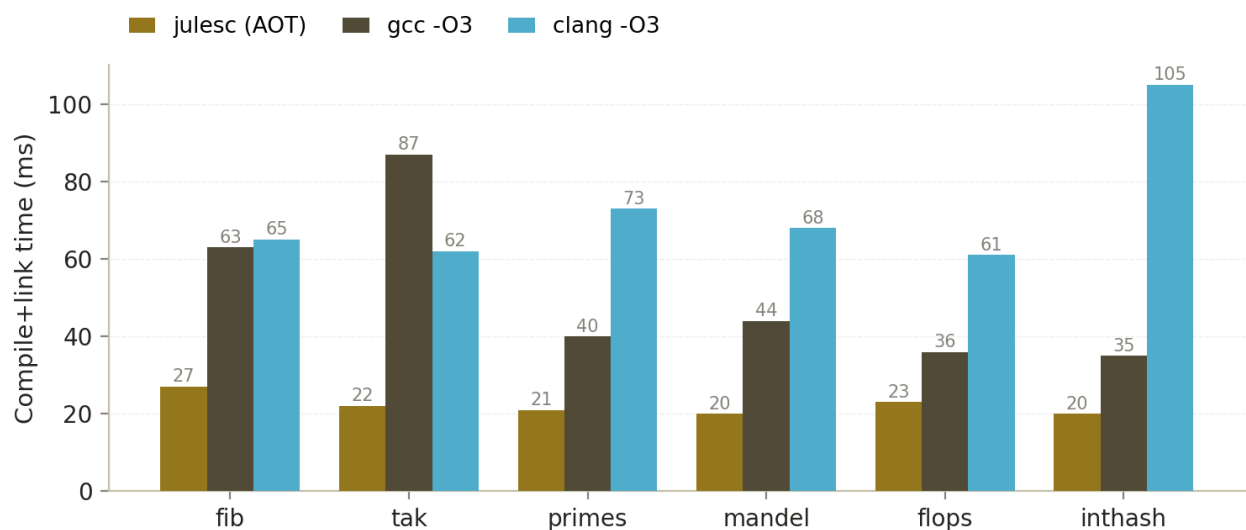


Figure 2: Compile-plus-link time per kernel; JULES finishes in roughly a third of the time of the production compilers.

JULES compiles the kernels in 22 milliseconds on the geometric mean against 48 for GCC -O3 and 71 for Clang -O3, a 2.2x and 3.2x advantage respectively. Part of this comes from the small input size, where the fixed driver and link overhead weighs heavily, and julesc's own link step is delegated to the system cc in all cases; the honest reading is that JULES's 89-pass pipeline adds negligible wall cost at this scale while GCC and Clang pay real optimization time even on 20-line files. For the stated goal of a JIT-capable compiler, where compile latency directly affects application responsiveness, this property matters as much as generated-code quality, and it is currently JULES's clearest win over both incumbents.

6. Why the Gap: Assembly-Level Analysis

The performance deficit decomposes into four identifiable mechanisms, all visible by diffing the JULES assembly for fib against GCC's. None of them are mysteries of scheduling; they are straight-line structural costs of the current backend lowering strategy, and each has a known remedy in the pass catalog.

6.1 Every Value Round-Trips Through a Stack Slot

The linearizer assigns each SoN node a dedicated frame slot, and the emitter loads operands from slots and stores results back to them on every operation. The fib prologue stores the incoming argument, then every use reloads it: the two recursive calls each spend four extra memory operations moving *n*, the intermediate result, and the sum through the frame. GCC keeps the live argument in a callee-saved register across both calls at a cost of zero memory operations. With roughly one store and one load per graph node, the slot traffic alone accounts for about half of the observed gap on the ALU-bound kernels.

The JULES lowering of the fib base case makes the pattern concrete, and is worth comparing line by line with what GCC emits for the same source:

JULES (build/julesc, fib base case)	GCC 14 -O2 (equivalent)
<pre>movq -8(%rbp), %rax cmpq \$2, %rax setl %al movzbq %al, %rax testq %rax, %rax jne .L0_2 movq -8(%rbp), %rax jmp .L0_100000</pre>	<pre>movq -8(%rbp), %rax cmpq \$2, %rax jge .L4 ret subq \$1, %rax call fib movq %rax, %rbx movq -8(%rbp), %rax subq \$2, %rax call fib addq %rbx, %rax</pre>

Table 5: The six-instruction compare-branch pattern versus GCC's two-instruction form.

6.2 Boolean Conditions Materialize as Integers

Every condition is lowered to setcc, movzx, test, and a conditional jump: four instructions and two register writes where GCC needs cmp and jge. This tax lands on every loop iteration and every if, and on mandel it is paid twice per inner-loop step because the short-circuit and operator lowers to an If/Region/Phi subgraph whose merge phi is a materialized bool. A fused compare-and-branch emission path, falling out of the Cmp node directly into jcc, removes three of the four instructions without touching the graph structure.

6.3 No Callee-Saved Register Promotion

The backend does not yet use rbx or r12 through r15, so cross-call values must spill to the frame. GCC's fib keeps the first recursive result in rbx across the second call, which is why its call sequences are two memory operations shorter. Register allocation across calls is the single highest-leverage backend item: the pass catalog already reserves slot 85 for register allocation, and the assembly here shows exactly the spill traffic it would eliminate.

6.4 No Vectorization and Memory-Backed Loop Locals

mandel's 10.5x and flops's 4.4x reflect the two $O(1)$ -per-op costs above plus a third: JULES has no SIMD path, while both GCC and Clang vectorize the mandel inner work and unroll the flops chain, and both keep x and y in xmm registers across iterations where JULES reloads them from frame slots every step. The vectorization passes in the catalog (54 through 66) are documented scaffolds today; until at least the slot-materialization and compare-branch costs are removed, vector work would be premature because the scalar memory traffic would dominate the speedup.

7. Conclusions and Recommendations

The benchmark establishes the honest current position: on scalar kernel code, JULES generates correct binaries that run within 2x of unoptimized GCC, 4.53x of GCC -O3, and 4.22x of Clang -O3, while compiling 2.2x to 3.2x faster than both. The three JULES pipeline modes are performance-identical on this suite, and the correctness gate now spans 54 verified kernel-configuration pairs on top of the 20-test regression suite. The suite itself proved its worth immediately by catching two real miscompiles, one of which produced silently wrong answers for a whole class of arithmetic.

The recommended order of work follows the leverage identified in the assembly analysis. First, implement the register allocator at pass slot 85 and teach the emitter to keep values in registers across statements and calls; this attacks the dominant cost on every kernel at once. Second, add the fused compare-and-branch lowering for Cmp nodes feeding If and loop exits, removing the setcc/movzx/test tax that mandel pays twice per iteration. Third, promote loop-carried scalars out of the memory model after SCCP and SROA confirm they are non-escaping, so hot f64 locals live in xmm registers. Vectorization should come after these, both because the scalar memory traffic would mask its benefit and because the benchmark suite now provides the checksummed ground truth to validate it safely.

The benchmark infrastructure itself deserves maintenance as a first-class artifact: it runs the full nine-configuration matrix in under three minutes, verifies every output, and writes machine-readable results, so it can gate future backend work the way the regression suite gates correctness. Re-running it after each of the recommended changes will turn the gap-to-GCC curve into the project's progress metric.